Instruction and Preference Fine-Tuning of a Small Language Model: An SFT → DPO Pipeline on Qwen3-0.6B

NLP with Deep Learning — Assignment 04
Track 1 (LLM Fine-Tuning), Option A: Standard Pipeline (SFT → DPO)

Abdullah Irfan 29266 | Ameer Hamza 28308 | Arhum Ali 29288

GitHub Repository

May 31, 2026

Abstract

We fine-tune the `Qwen3-0.6B-Base` model for instruction following through the standard two-stage alignment pipeline: supervised fine-tuning (SFT) on a curated subset of `UltraChat-200k`, followed by Direct Preference Optimization (DPO) on `UltraFeedback-binarized`. Across five SFT and five DPO Low-Rank Adaptation (LoRA) trials we systematically vary adapter rank, target modules, learning rate, effective batch size, epochs, and the DPO temperature β, so that the ten trials collectively sweep a wide configuration space. Every checkpoint is evaluated on a fixed 15-prompt, three-tier test set with a comprehensive metric suite—BLEU, chrF, ROUGE-L, and BERTScore Precision/Recall/F_1—complemented by validation loss and, for DPO, reward accuracy and reward margin. The best SFT model (**sft4**) raises mean BERTScore-F_1 from -0.16 (base) to 18.47 and BLEU from 6.90 to 14.40; the best preference-tuned model (**dpo2**) improves these further to 22.88 and 14.69 while producing markedly more concise, better-structured, and safer responses. We explain the reasoning behind every design choice, analyse the effect of each hyperparameter, document failure modes (degenerate repetition and DPO reward over-optimization), report full resource/timing data, and provide a reproducible recipe.

Contents

1 Introduction and Task Overview **5**

2 Platform, Environment, and Reproducibility **5**

3 Methodology **6**

3.1 Base Model and Tokenizer 6
3.2 Why LoRA (Parameter-Efficient Fine-Tuning) 7
3.3 The DPO Objective 7
3.4 Evaluation Test Set and Gold References 7
3.5 Evaluation Metrics and Why We Use Each 8

4 Datasets and Preprocessing **8**
4.1 SFT Data: UltraChat-200k — and Why . . . 8
4.2 DPO Data: UltraFeedback-binarized — and Why . . . 9
4.3 Justification of Subset Sizes . . . 10

5 Baseline: The Un-tuned Base Model **10**

6 Stage 1: Supervised Fine-Tuning **11**
6.1 Experimental Design and the Hypothesis Behind Each Trial . . . 11
6.2 Results . . . 11
6.3 Model Selection — and the Reasoning . . . 12
6.4 Analysis: What the Trials Teach Us . . . 13

7 Stage 2: Direct Preference Optimization **15**
7.1 Results . . . 15
7.2 Model Selection . . . 16
7.3 Analysis: Reward Optimisation vs. Generation Quality . . . 16

8 Comparative Analysis: Base vs. SFT vs. DPO **17**
8.1 Quantitative Progression . . . 17
8.2 Qualitative Comparison . . . 19
8.3 Preference-Specific Showcase (Safety / Helpfulness / Tone) . . . 19

9 Discussion **19**
9.1 Impact of Each Hyperparameter . . . 19
9.2 Strengths and Weaknesses; When Each Excels . . . 20
9.3 Common Failure Cases and Unexpected Behaviours . . . 20

10 Resource Usage and Training Time **20**

11 Limitations and Future Work **21**

12 Conclusion **22**

References **22**

A Reproducibility Checklist **22**

B Additional Sample Outputs (Best DPO Model, `dpo2`) **23**

List of Figures

1 Base model per-prompt BLEU and BERTScore-F_1. Behaviour is highly uneven: a handful of prompts score well while many collapse into prompt-echoing or degenerate repetition (negative BERTScore). 11

2 SFT trials across the full metric suite. `sft4` and `sft5` lead on every surface metric; `sft2` suffers a BERTScore-F_1 collapse caused by verbose, repetitive generations despite a competitive BLEU. 12

3 SFT selection. Bars: combined score (red = selected `sft4`); black line: validation loss, used as the tie-breaker between `sft4` and `sft5`. 13

4 SFT: LoRA capacity (% trainable) vs. combined score (blue) and final training loss (red). Lower training loss at higher capacity does *not* translate into better generations—combined score peaks at the moderate `sft4`. 13

5 SFT training-loss curves. All trials converge smoothly; multi-epoch / high-rank runs reach lower training loss without a matching gain in held-out quality. 14

6 SFT BLEU (left) and BERTScore-F_1 (right) by trial × difficulty tier. Improvements concentrate on easy/medium prompts; the harder tier remains hard. `sft2`'s negative medium BERTScore is its repetition failure. 14

7 DPO trials across the metric suite. `dpo2` (low LR, β=0.1) is best on every surface metric; `dpo1` (same β but high LR) collapses despite a large reward margin. . . . 16

8 DPO training dynamics. Loss falls and the reward margin grows for all β; the high-LR `dpo1` is the most volatile and, despite its large margin, generalises worst (cf. Table 8). 17

9 DPO reward accuracy (left axis) and reward margin (right axis) per trial. All trials exceed 0.64 accuracy; margin grows with β/LR but does *not* predict generation quality. 17

10 Progression across stages. SFT provides the first large gain; DPO adds a further semantic improvement, strongest on the easy/medium tiers (Figure 10b). 18

11 Average response length. SFT lengthens responses (learning to elaborate); the best DPO model tightens them back to base-like brevity while remaining on-topic. 18

12 Per-trial training time (left) and peak GPU memory (right). The dotted line marks the ≈15 GB practical T4 ceiling; all trials stay well below it. 21

List of Tables

1 Experimentation platform and software environment. 6

2 Composition of the 15-prompt evaluation set (three difficulty tiers). 8

3 SFT preprocessing pipeline and resulting statistics (UltraChat-200k). 9

4 DPO preprocessing pipeline and statistics (UltraFeedback-binarized). 9

5 SFT trial configurations and the design question each one tests. "all-linear" = `q,k,v,o,gate,up,down`. 11

6 SFT results: comprehensive metrics, validation loss, and resource usage. Best combined score in **bold**. (B-P/R/F1 = BERTScore Precision/Recall/F_1.) 12

7 DPO trial configurations (LoRA fixed at r=16 on q,k,v,o, 0.764% trainable). . . . 15

8 DPO results: reward statistics, generation metrics, and resources. Best combined score in **bold**. 15
9 Stage-by-stage comparison on the 15-prompt test set (best model per stage). . . . 18
10 Representative outputs across stages (trailing garbled tokens trimmed; "[. . .]" marks truncated repetition). 19
11 Parameters of the two selected (best) models. 21
12 Further `dpo2` outputs illustrating successes and failure modes. 23

1 Introduction and Task Overview

A pre-trained *base* language model is optimised for a single objective: predict the next token in ordinary text. This makes it an excellent text *continuer* but a poor *assistant*. Asked "Which planet is known as the Red Planet?", a base model is just as likely to continue with another plausible-looking question as to answer, because both are statistically reasonable continuations of web text. Turning such a model into a helpful assistant requires **alignment**: explicitly teaching it (i) the *format* of instruction following and (ii) human *preferences* about what a good answer looks like.

Why a two-stage SFT → DPO pipeline? The two needs above map onto two distinct training signals and therefore two stages:

1. **Supervised Fine-Tuning (SFT)** uses *demonstrations* (prompt → ideal response). It is a maximum-likelihood objective that teaches the model the response format and stops it from echoing the prompt. SFT is cheap and stable but can only imitate; it cannot learn that one valid answer is *better* than another.
2. **Direct Preference Optimization (DPO)** uses *comparisons* (for the same prompt, a *chosen* answer that humans preferred and a *rejected* one). It directly optimises the model to assign higher likelihood to preferred responses, capturing qualities—helpfulness, harmlessness, concision, tone—that demonstrations alone do not convey.

DPO is chosen over classical RLHF/PPO because it reaches comparable preference alignment *without* training a separate reward model or running an unstable on-policy RL loop: it reformulates the RLHF objective into a simple classification-style loss on preference pairs (Section 3.3), which is far better suited to a 40-hour assignment on a single GPU.

What this report delivers. Following the assignment, we (a) baseline the raw base model, (b) run and analyse five SFT LoRA trials and select the best by BLEU+BERTScore (validation loss as tie-breaker), (c) run five DPO trials from that best SFT model and select the best the same way, and (d) compare base vs. instruction-tuned vs. preference-tuned behaviour both quantitatively and qualitatively. Section 2 covers the platform; Section 3 the methodology and the theory behind LoRA and DPO; Section 4 the data and preprocessing; Sections 5–7 the three experimental stages; Section 8 the comparison; Sections 9–11 discussion, resources, and limitations.

2 Platform, Environment, and Reproducibility

All experiments ran on **Kaggle Notebooks** with a single NVIDIA **Tesla T4** GPU (16 GB). Table 1 lists the exact stack. Below we justify the non-obvious environment decisions, because each materially affected whether training succeeded and how fast it ran.

Why a single GPU (and not the free T4×2)? Loading the model with `device_map="auto"` on a two-GPU instance *shards* the network across both cards. That mode is designed for *inference*; during *training* every step must shuttle activations between GPU 0 and GPU 1 over PCIe, which both slows each step to a crawl and intermittently *deadlocks* mid-epoch. A 0.6B model fits comfortably on one T4, so we pin training to a single device with `CUDA_VISIBLE_DEVICES=0` and load with `.to("cuda")`. This removed the stalls entirely and sped up training several-fold.

Why fp16 and not bf16? The T4 is a Turing-generation card: it has hardware *fp16* tensor cores but *no* native *bf16* support, so bf16 is emulated and runs roughly 2× slower. We therefore train in fp16. The only cost is slightly lower numerical stability, which we mitigate with gradient clipping (max-norm 1.0) and the optimiser's automatic loss scaling; no divergence was observed.

Why a 512-token context? The dominant memory term for a causal LM is the output logits tensor of shape (batch × sequence × vocabulary). Qwen3's vocabulary is very large (151,936), so logits memory grows quickly with sequence length. Capping at 512 tokens keeps peak memory well under 16 GB while still covering the vast majority of single-turn instructions in our data (median ≈328 tokens, Section 4).

Why a fixed seed and greedy decoding? A global seed of **42** (set for `set_seed`, Python `random`, and NumPy) plus greedy decoding (`do_sample=False`) make every metric exactly reproducible, so differences between trials reflect configuration, not sampling noise.

Table 1: Experimentation platform and software environment.

Component	Setting
Platform	Kaggle Notebooks (Internet on), single NVIDIA T4 (16 GB)
Base model	`Qwen/Qwen3-0.6B-Base` (≈596M params, 151,936 vocab)
Precision	fp16 (T4 tensor cores); `use_cache=False` during training
PEFT method	LoRA (`peft`), dropout 0.05, bias `none`
Frameworks	`transformers` ≥4.51, `trl` ≥0.15, `peft` ≥0.14, `datasets` ≥3.2, `accelerate` ≥1.3
Metrics libraries	`sacrebleu`, `bert-score`, `rouge-score`
Max sequence length	512 tokens (SFT and DPO); DPO max prompt length 256
Optimiser / schedule	AdamW, cosine schedule, warmup 0.03–0.05, grad-clip 1.0
Memory flag	`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`
Global seed	42

Reproducibility. The three notebooks (`00_baseline_eval`, `01_sft_trials`, `02_dpo_trials`) are self-contained and emit every artefact used here: CSV metric tables, JSON preprocessing statistics, full training logs, sample-output Markdown, and the best LoRA adapter. All figures in this report are regenerated deterministically from those files by the included `make_figures.py`.

3 Methodology

3.1 Base Model and Tokenizer

We chose `Qwen3-0.6B-Base` over the other suggested option, TinyLlama-1.1B, for four reasons. **(i) Compute fit:** at 0.6B parameters it leaves ample headroom on a 16 GB T4 for LoRA optimiser state, activations, and the large-vocabulary logits, which is essential because we must run *ten* trials in total within Kaggle session limits. **(ii) Quality head-room:** Qwen3 is a strong, modern architecture, so measured gains reflect genuine instruction learning rather than a weak backbone artificially inflating the apparent benefit of fine-tuning. **(iii) A clean base checkpoint:** the assignment requires starting from a *base* (non-instruct) model so that SFT and DPO have a measurable effect; an already-instruction-tuned model would mask the contribution of each stage. **(iv) Tokenizer maturity:** Qwen3's byte-level BPE tokenizer handles code, punctuation, and multilingual text robustly.

Chat template. We attach a **ChatML** template (`<|im_start|>`role … `<|im_end|>`) to delimit system, user, and assistant turns. A consistent template is critical: the model learns to emit assistant content only after the assistant tag and to stop at `<|im_end|>`. Because the base tokenizer has no padding token, we set `pad_token = eos_token`. The *same* template is used in

all three notebooks so that baseline, SFT, and DPO evaluations are strictly comparable.

3.2 Why LoRA (Parameter-Efficient Fine-Tuning)

Full fine-tuning of every weight is unnecessary and, on a single T4, impractical (it would require storing full-precision gradients and optimiser moments for all 596M parameters). **LoRA** instead freezes the pre-trained weights W_0 and learns a low-rank update $\Delta W = BA$, where $A \in R^{r\times d}$ and $B \in R^{d\times r}$ with rank $r \ll d$. Only A and B are trained, so the trainable footprint drops to a small fraction of the model (here 0.19%–6.34%). The scaling factor α controls the magnitude of the update (ΔW is scaled by α/r). This yields three benefits we rely on: tiny memory/compute cost, tiny (∼tens of MB) checkpoints that are easy to store and ship between notebooks, and reduced over-fitting on small data. The two LoRA knobs we sweep are **rank** r (capacity of the update) and **target modules** (which projections receive an adapter—attention only, or all linear layers including the MLP).

3.3 The DPO Objective

DPO optimises the same goal as RLHF—make the policy prefer human-preferred responses—but eliminates the reward model and RL loop. Given a prompt x with a chosen response y_w and rejected response y_l, DPO minimises

$$\mathcal{L}_{\text{DPO}} = -\,E_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right],$$

where π_θ is the trained policy, π_{ref} the frozen reference (our merged SFT model), σ the logistic function, and β a temperature. Intuitively, the model is rewarded for raising the *relative* log-probability of the chosen over the rejected response. **Why β matters:** it controls how far the policy may move from the reference. Small β permits large updates (risking degeneration); large β keeps the policy close to the SFT model (risking under-fitting of preferences). Because π_{ref} is just the frozen base of the same LoRA-wrapped model, DPO needs *no* extra network. The natural diagnostics are **reward accuracy** (how often the chosen response gets the higher implicit reward) and **reward margin** (by how much).

3.4 Evaluation Test Set and Gold References

The assignment requires evaluating the same prompts at every stage, so we authored a fixed set of **15 prompts** (exceeding the 10-prompt minimum) across three difficulty tiers and many task types (Table 2). **Why more than 10 and why tiered?** A larger, tiered set gives a finer, more discriminative picture—we can see *where* each model improves (easy vs. hard) instead of a single averaged number. **Why short, unambiguous gold answers?** Lexical metrics (BLEU/ROUGE) compare against references; short canonical answers make "correct" responses score high and rambling or off-topic ones score low, which is the behaviour we want to measure. Three additional *unscored* showcase prompts (safety, helpfulness, tone) are reserved for qualitative DPO analysis, where preference gains do not surface in n-gram overlap.

Table 2: Composition of the 15-prompt evaluation set (three difficulty tiers).

Tier	# Prompts	Task types
Easy	3	factual recall, binary sentiment, single-step arithmetic
Medium	9	constrained list, tone rewrite, grounded summarization, span extraction, 3-class sentiment, unit conversion, code one-liner, grammar, translation
Harder	3	multi-step date reasoning, multi-step arithmetic, length-constrained explanation

3.5 Evaluation Metrics and Why We Use Each

No single automatic metric captures answer quality, so we report a complementary suite:

- **BLEU** (sacreBLEU): n-gram *precision*. Rewards exact lexical overlap; good for short canonical answers but harsh on valid paraphrases.
- **chrF**: character n-gram F-score. More forgiving of morphology and very short answers than BLEU, so it complements BLEU's brittleness.
- **ROUGE-L**: longest-common-subsequence *recall*. Sensitive to whether the response *covers* the reference content, not just matches n-grams.
- **BERTScore ($\mathbf{P}/\mathbf{R}/\mathbf{F_1}$)**: contextual-embedding similarity using `roberta-large`, *rescaled with the baseline*. It captures *semantic* adequacy that surface metrics miss (a correct paraphrase scores well even with low n-gram overlap). *Why values can be negative:* baseline rescaling subtracts the score of a random pairing, so a response that is no more similar to the reference than random text receives a negative number—this is expected and is precisely how we detect degenerate/off-topic outputs.

Why a combined selection score? To balance lexical and semantic quality in a single, assignment-mandated ranking we use combined = BLEU + BERTScore-F_1, with **lower validation loss** as the tie-breaker when two trials are within one point. Validation loss is reported for every trial but is used only as a tie-breaker because, as we show in Section 6, loss is a weak proxy for generation quality.

4 Datasets and Preprocessing

4.1 SFT Data: UltraChat-200k — and Why

We use `HuggingFaceH4/ultrachat_200k` for SFT. **Why this dataset?** It is a large, high-quality, diverse, ChatGPT-distilled instruction corpus that is *not* used in class, satisfying the assignment, and its broad topic coverage teaches general instruction following rather than one narrow skill. We keep only the *first* user→assistant turn (**why:** single-turn data matches our single-turn evaluation and keeps sequences short enough for the 512-token budget) and render it through ChatML. The preprocessing pipeline (Table 3) is logged to `sft_preprocessing_stats.json`; each step has a purpose:

- **Whitespace normalization** — collapse runs of spaces/newlines so tokenization is consistent and not wasted on formatting noise.

- **Minimum response length (20 chars)** — drop near-empty answers that teach the model nothing useful.
- **Prompt de-duplication** — avoid over-weighting repeated prompts, which would bias the model and inflate apparent learning.
- **Token-length windowing** $[16, 512]$ — discard too-short pairs (little signal) and too-long pairs (would be truncated, corrupting the target). This is the largest filter (it removes the long multi-turn-style examples) and is what keeps every training sequence within the memory budget.

Table 3: SFT preprocessing pipeline and resulting statistics (UltraChat-200k).

Step	Operation	Train	Val
0	Raw candidates sampled (seed 42)	6,000	900
1	Dropped: malformed turn structure	0	0
2	Dropped: response <20 characters	2	1
3	Dropped: duplicate prompt (lowercased prefix)	0	0
4	Dropped: token length outside $[16, 512]$	1,532	250
5	**Kept (final)**	**2,000**	**300**
Token length of kept set: median / mean / max		328 / 324.8 / 512	323 / 314.2 / 511

4.2 DPO Data: UltraFeedback-binarized — and Why

For preference optimisation we use `HuggingFaceH4/ultrafeedback_binarized`, which provides (*chosen*, *rejected*) completion pairs with numeric quality scores. **Why this dataset?** It is the canonical binarised preference set aligned with our SFT domain, so the preferences are about the same kind of instruction following the model already learned. Beyond basic cleaning we add a **preference-margin filter**: keep a pair only if $\text{score}_{\text{chosen}} - \text{score}_{\text{rejected}} \geq 0.5$. **Why filter by margin?** DPO learns from the *contrast* within each pair; pairs whose chosen and rejected answers are nearly tied carry little signal and inject label noise, which can destabilise training. Filtering raises the mean retained margin to 2.06, giving cleaner gradients (Table 4). We also drop identical pairs (zero signal) and too-short completions.

Table 4: DPO preprocessing pipeline and statistics (UltraFeedback-binarized).

Step	Operation	Train	Val
0	Raw candidates sampled (seed 42)	6,000	900
1	Dropped: completion <20 characters	176	37
2	Dropped: identical chosen/rejected	3	0
3	Dropped: margin < 0.5 (weak preference signal)	257	49
4	Dropped: duplicate prompt	3	0
5	**Kept (final)**	**2,000**	**300**
Preference margin of kept set: mean / median		2.06 / 1.50	

4.3 Justification of Subset Sizes

The assignment explicitly permits subsets "as appropriate for your compute budget." We cap both stages at **2,000 training** and **300 validation** examples. **Why this size?** Three reasons: (i) it is empirically sufficient for LoRA to learn the instruction format—the large metric gains in Section 8 confirm the model generalises to unseen test prompts; (ii) it keeps each of the ten trials to minutes (SFT) or tens of minutes (DPO), so the full five-trial sweep per stage fits inside Kaggle's session limit with margin for evaluation and checkpointing; and (iii) in pilot runs, larger subsets produced diminishing returns relative to their wall-clock cost. We sample 3× the target before filtering so that, after the length/margin filters remove a large fraction, exactly 2,000/300 clean examples remain.

5 Baseline: The Un-tuned Base Model

The raw `Qwen3-0.6B-Base` scores a mean **BLEU of 6.90** and a mean **BERTScore-$\mathbf{F_1}$ of** -0.16 across the 15 prompts. The negative semantic score is not a bug—it quantifies how badly an un-aligned base model behaves. We observe three recurring failure modes:

1. **Prompt echoing / topic drift:** instead of answering, the model continues the prompt or invents a related question (the "Red Planet" prompt is answered with a question about the capital of France).
2. **Degenerate repetition:** many responses collapse into endless repeated tokens or symbols (the unit-conversion and CPU prompts), a classic next-token failure under greedy decoding.
3. **Occasional accidental success:** a few prompts whose answer is essentially a continuation of the input (e.g. grounded summarization, prompt 6) score well, which is why the average is not uniformly catastrophic.

Figure 1 shows this strongly bimodal per-prompt behaviour. Aggregated by tier, mean BERTScore is -5.0 (easy), $+12.5$ (medium), and -33.2 (harder): the model is *worst* on the hardest prompts, exactly where reasoning is required. This baseline is the reference against which both fine-tuning stages are measured.

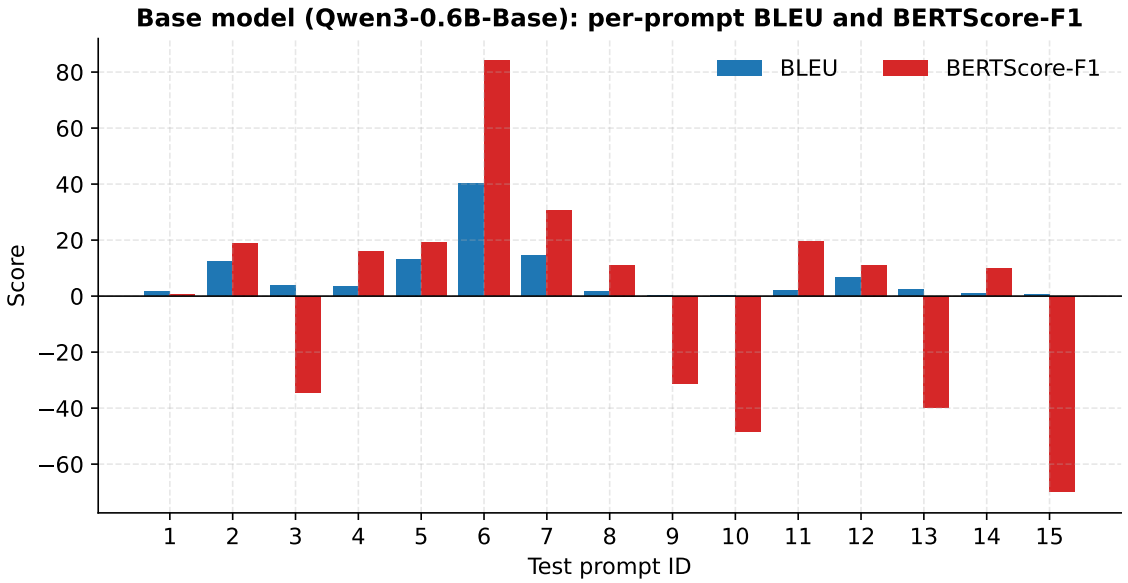


Figure 1: Base model per-prompt BLEU and BERTScore-F_1. Behaviour is highly uneven: a handful of prompts score well while many collapse into prompt-echoing or degenerate repetition (negative BERTScore).

6 Stage 1: Supervised Fine-Tuning

6.1 Experimental Design and the Hypothesis Behind Each Trial

The five SFT trials (Table 5) are not random—each probes a specific question, and together they sweep LoRA rank ($8 \to 64$), target modules (attention-only $\to$ all-linear), learning rate ($5\times10^{-5} \to 3\times10^{-4}$), effective batch ($4 \to 16$), and epochs ($1 \to 3$), so trainable parameters span 0.19%–6.34%.

Table 5: SFT trial configurations and the design question each one tests. "all-linear" = `q,k,v,o,gate,up,down`.

Trial	r/α	Targets	LR	bs×ga	ep.	Train.%	Tests...
sft1	8/16	q,v	2e-4	4×1	1	0.192	minimal adapter baseline
sft2	16/32	q,k,v,o	2e-4	4×2	2	0.764	more attention + 2 epochs
sft3	32/64	all-linear	1e-4	2×4	2	3.276	MLP adapters, lower LR
sft4	16/32	all-linear	3e-4	4×4	1	1.665	higher LR, large batch, 1 epoch
sft5	64/128	all-linear	5e-5	2×8	3	6.343	max capacity, many epochs

6.2 Results

Full results are in Table 6 and Figure 2. *Every* trial dramatically improves over the base model, confirming that even a small LoRA on 2,000 examples teaches instruction following. The two strongest trials, **sft4** and **sft5**, reach near-identical combined scores (32.87 vs. 32.37) with BERTScore-$F_1 \approx 18$ and BLEU ≈ 14.3.

Table 6: SFT results: comprehensive metrics, validation loss, and resource usage. Best combined score in **bold**. (B-P/R/F1 = BERTScore Precision/Recall/F_1.)

Trial	Tr. loss	Val. loss	BLEU	chrF	ROUGE-L	B-P	B-R	B-F1	Comb.	Words	Time(s)
sft1	1.634	1.638	10.16	27.49	22.50	-11.26	46.58	15.95	26.11	118.3	396
sft2	1.621	1.628	11.25	18.84	24.35	-59.68	42.99	-14.85	-3.60	92.4	825
sft3	1.554	1.628	8.03	26.01	26.19	-24.35	45.87	7.76	15.79	96.4	890
sft4	1.654	1.630	**14.40**	35.78	33.56	-8.81	49.95	**18.47**	**32.87**	80.2	483
sft5	1.484	1.637	14.25	36.94	35.11	-10.95	52.22	18.12	32.37	60.4	1322

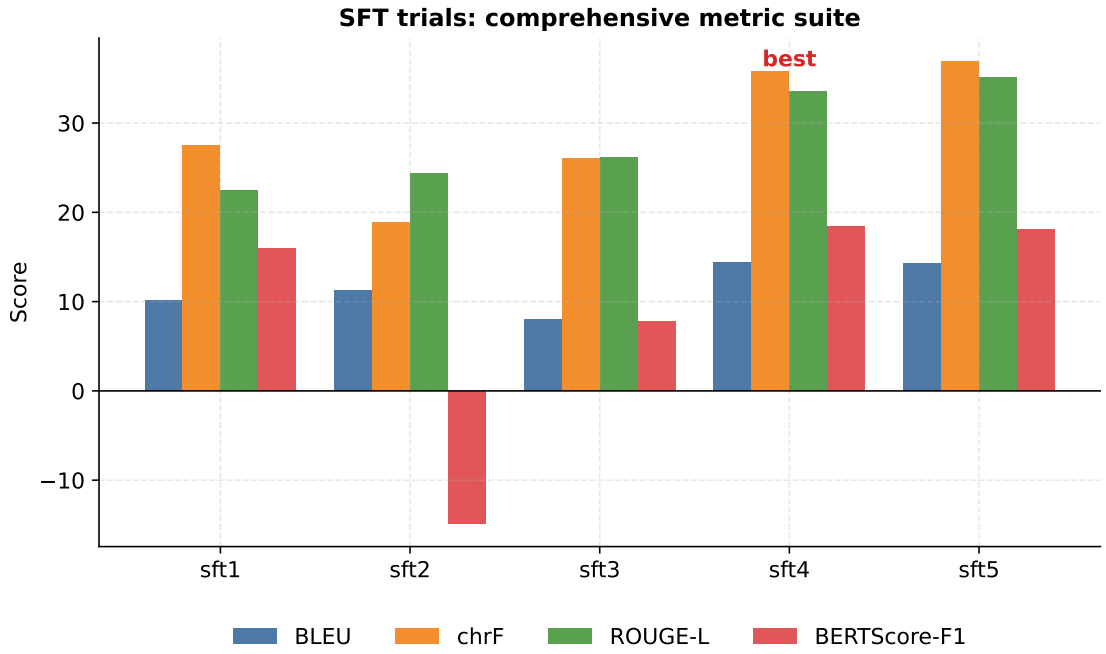


Figure 2: SFT trials across the full metric suite. `sft4` and `sft5` lead on every surface metric; `sft2` suffers a BERTScore-F_1 collapse caused by verbose, repetitive generations despite a competitive BLEU.

6.3 Model Selection — and the Reasoning

Per the assignment rule we rank by combined score and break ties (within one point) by validation loss. As Figure 3 shows, `sft4` (32.87) and `sft5` (32.37) differ by only 0.5 points, so the tie-breaker applies: `sft4` has the lower validation loss (1.630 vs. 1.637) and is **selected** as the SFT model carried into DPO. **Why this is also the sensible engineering choice:** `sft4` matches `sft5`'s quality using *one* epoch instead of three and roughly one-sixth the trainable parameters, training in 8.1 min vs. 22.0 min—more quality per unit compute.

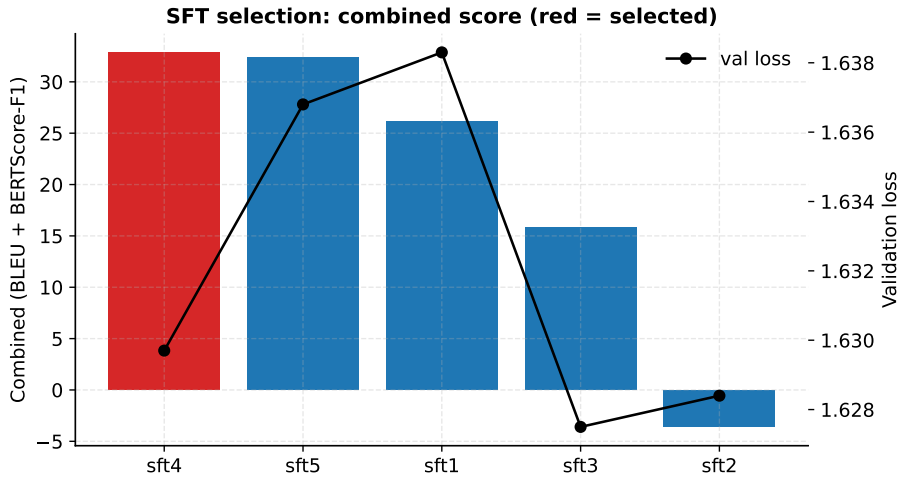


Figure 3: SFT selection. Bars: combined score (red = selected `sft4`); black line: validation loss, used as the tie-breaker between `sft4` and `sft5`.

6.4 Analysis: What the Trials Teach Us

Capacity does not buy quality (the key insight). Figure 4 plots trainable % against both combined score and final training loss. Training loss falls *monotonically* with capacity—`sft5` (6.34% trainable, 3 epochs) fits the training data best (loss 1.484)—yet generation quality *peaks at the moderate* `sft4` and even collapses for `sft2`. **Why:** lower training loss means the adapter memorises the demonstrations better, but held-out instruction following depends on *generalisation* and decoding behaviour, not training-set likelihood. This is why we use combined score (not loss) for selection.

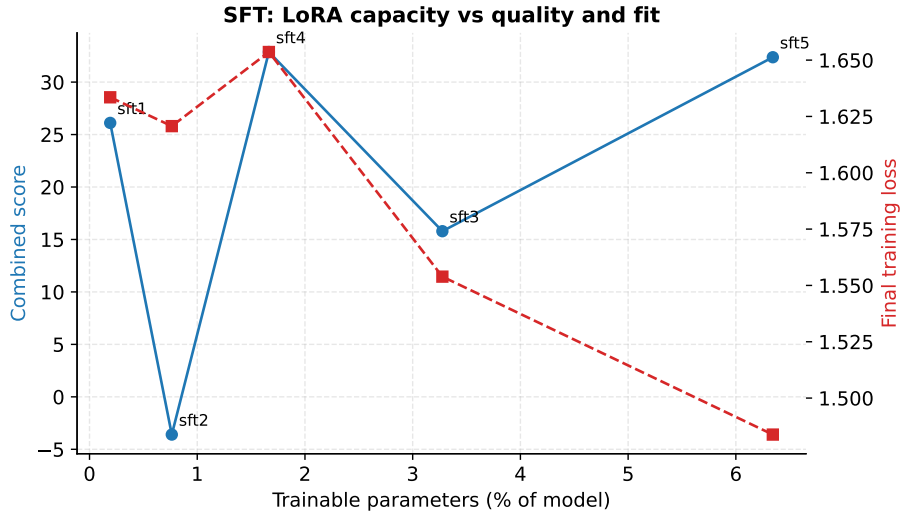


Figure 4: SFT: LoRA capacity (% trainable) vs. combined score (blue) and final training loss (red). Lower training loss at higher capacity does *not* translate into better generations—combined score peaks at the moderate `sft4`.

Learning rate is the stability knob. `sft2`'s BERTScore collapse (Table 6) comes from verbose, repetitive output; the mid-range LR of `sft4` (3×10^{-4}) is the sweet spot between under-fitting and degeneration. **Target modules help, up to a point:** extending adapters to the MLP (all-linear, `sft3`/`sft4`/`sft5`) generally beats attention-only (`sft1`), but beyond `sft4`'s moderate

rank the extra capacity is wasted.

Training dynamics. The loss curves (Figure 5) are smooth and stable for all trials; multi-epoch / high-rank runs reach visibly lower loss, consistent with the capacity analysis.

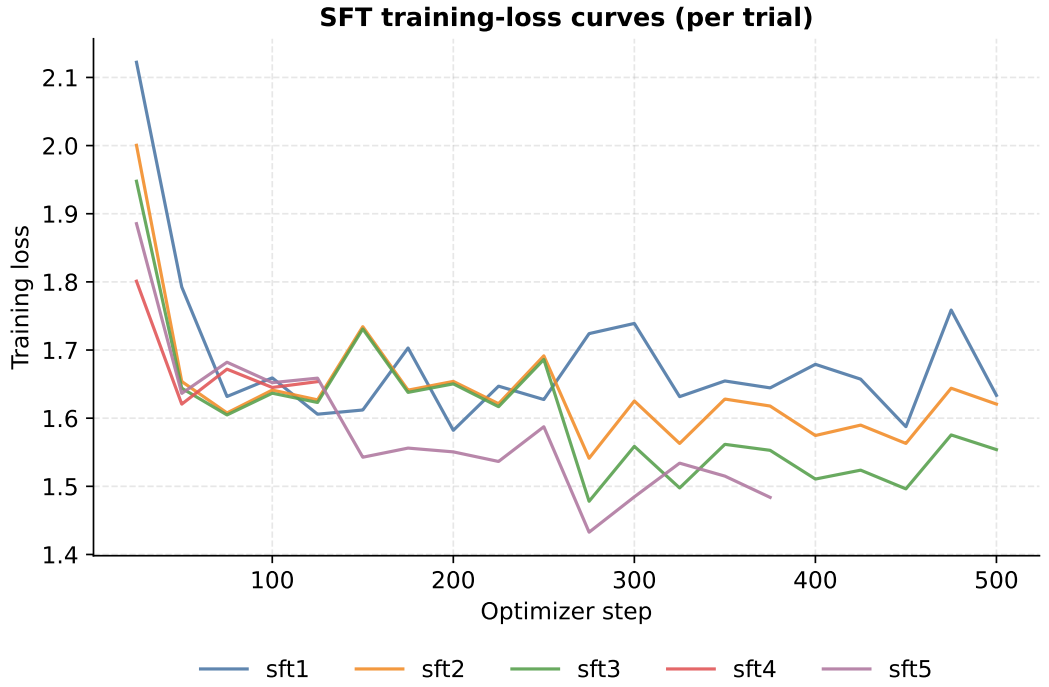


Figure 5: SFT training-loss curves. All trials converge smoothly; multi-epoch / high-rank runs reach lower training loss without a matching gain in held-out quality.

Where the gains land. The difficulty heatmap (Figure 6) shows improvements concentrate on *easy* and *medium* prompts—`sft4` attains the best easy-tier BERTScore (31.3)—while the *harder* multi-step reasoning tier stays difficult for all trials (BLEU ≈ 1.5). **Why:** a 0.6B model has limited reasoning capacity, and SFT teaches *format*, not new reasoning ability; this gap is part of the motivation for the preference stage.

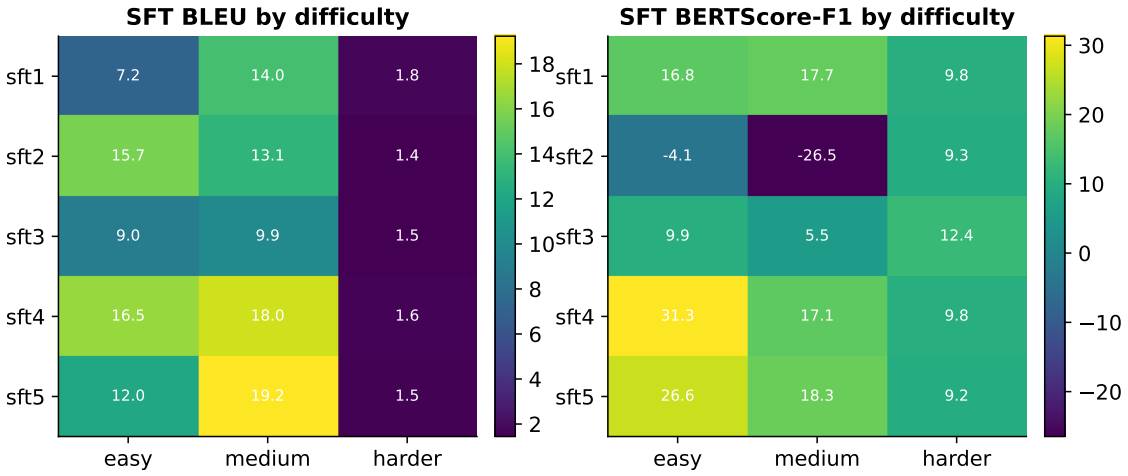


Figure 6: SFT BLEU (left) and BERTScore-F_1 (right) by trial × difficulty tier. Improvements concentrate on easy/medium prompts; the harder tier remains hard. `sft2`'s negative medium BERTScore is its repetition failure.

7 Stage 2: Direct Preference Optimization

We merge the `sft4` LoRA into the base weights to form the DPO starting policy and its frozen reference, then run five DPO trials. **Why hold LoRA fixed here?** The assignment asks the DPO sweep to vary the *preference* hyperparameters; fixing the adapter (rank 16 on `q,k,v,o`, 0.76% trainable) isolates the effect of β, learning rate, batch size, and epochs (Table 7) instead of confounding them with adapter capacity. DPO needs no separate reward model (Section 3.3).

Table 7: DPO trial configurations (LoRA fixed at r=16 on q,k,v,o, 0.764% trainable).

Trial	β	LR	bs×ga (eff.)	epochs	Tests...
dpo1	0.10	5e-5	1×4 (4)	1	moderate β, *high* LR
dpo2	0.10	1e-5	1×8 (8)	1	moderate β, *low* LR
dpo3	0.30	5e-6	1×8 (8)	2	higher β, very low LR, 2 ep.
dpo4	0.05	5e-5	1×16 (16)	1	low β + high LR (aggressive)
dpo5	0.50	1e-6	1×16 (16)	2	high β (stay near reference)

7.1 Results

Table 8 and Figures 7–9 summarise the outcomes. **All five trials achieve a reward accuracy above** 0.64, confirming DPO successfully learns to rank chosen over rejected completions. Crucially, however, reward optimisation and *generation* quality diverge.

Table 8: DPO results: reward statistics, generation metrics, and resources. Best combined score in **bold**.

Trial	Val. loss	Rew. acc	Rew. marg	BLEU	chrF	ROUGE-L	B-F1	Comb.	Words	Time(s)
dpo1	0.621	0.663	0.873	8.10	20.06	16.87	0.89	8.99	94.8	1242
dpo2	0.603	0.657	0.307	**14.69**	38.60	35.42	**22.88**	**37.56**	55.9	1244
dpo3	0.571	0.677	0.556	14.55	37.04	34.52	19.53	34.08	68.9	2382
dpo4	0.608	0.643	0.452	10.76	30.17	26.15	11.12	21.88	71.6	1244
dpo5	0.627	0.650	0.173	14.30	35.85	33.77	18.19	32.49	76.6	2382

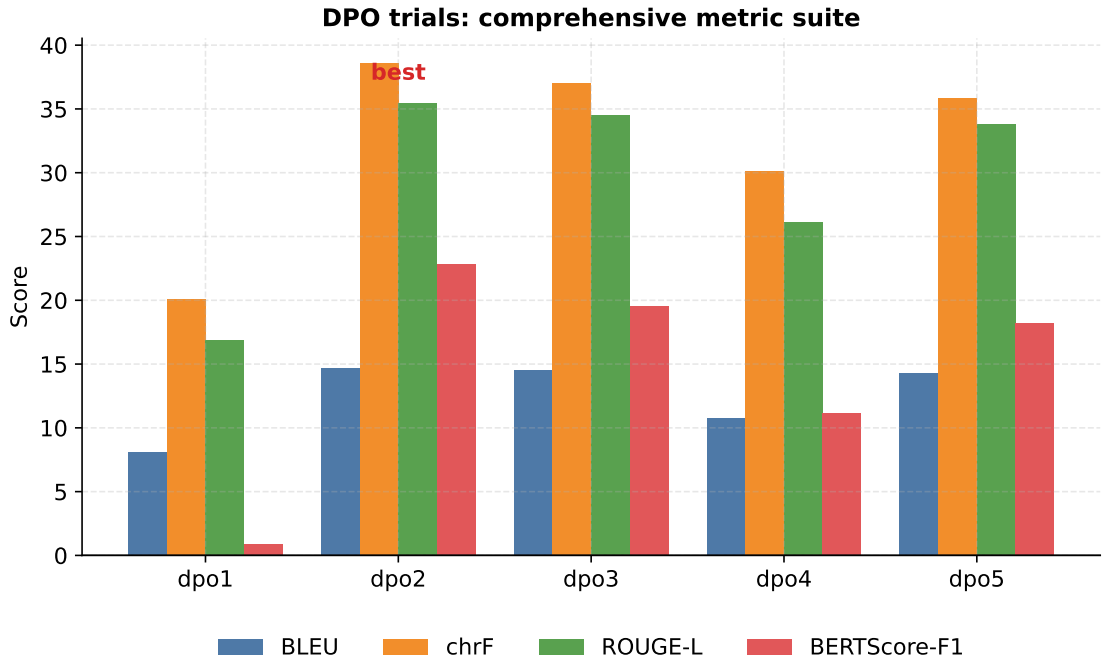


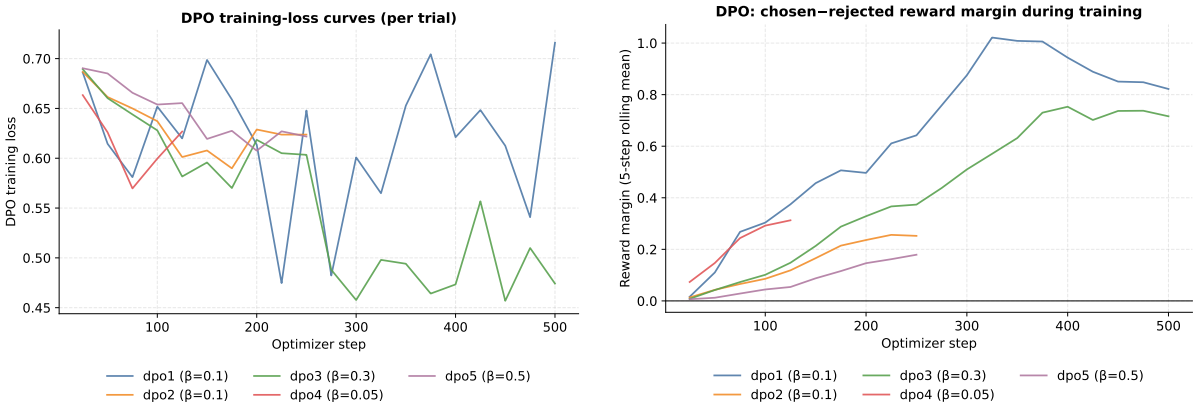
Figure 7: DPO trials across the metric suite. `dpo2` (low LR, β=0.1) is best on every surface metric; `dpo1` (same β but high LR) collapses despite a large reward margin.

7.2 Model Selection

Ranking by combined score, **dpo2** (37.56) leads `dpo3` (34.08) by more than three points, so no tie-break is needed; `dpo2` also has a low validation loss (0.603). It is **selected** as the final SFT+DPO model; its parameters are in Table 11.

7.3 Analysis: Reward Optimisation vs. Generation Quality

The most instructive finding is an **anti-correlation between reward margin and text quality at high learning rates**. `dpo1` (LR 5×10^{-5}) attains the *largest* reward margin (0.873) yet the *worst* generations (BERTScore-F_1 0.89, combined 8.99). **Why:** a high learning rate lets the policy chase the preference signal so hard that it drifts far from the fluent SFT reference and degenerates—a textbook case of DPO *reward over-optimisation*. The winning `dpo2` uses a 5× smaller LR: a modest margin (0.307) but the best text. **The role of β:** moderate β=0.1 (`dpo2`) balances movement and stability; very large β=0.5 (`dpo5`) over-constrains the policy to the reference (tiny margin 0.173, smaller gains); low β with high LR (`dpo4`) is the most aggressive and underperforms. Figures 8a–8b show loss decreasing and the reward margin growing during training, with the high-LR `dpo1` the most volatile.



(a) DPO training loss. **(b)** Reward margin (chosen−rejected).

Figure 8: DPO training dynamics. Loss falls and the reward margin grows for all β; the high-LR `dpo1` is the most volatile and, despite its large margin, generalises worst (cf. Table 8).

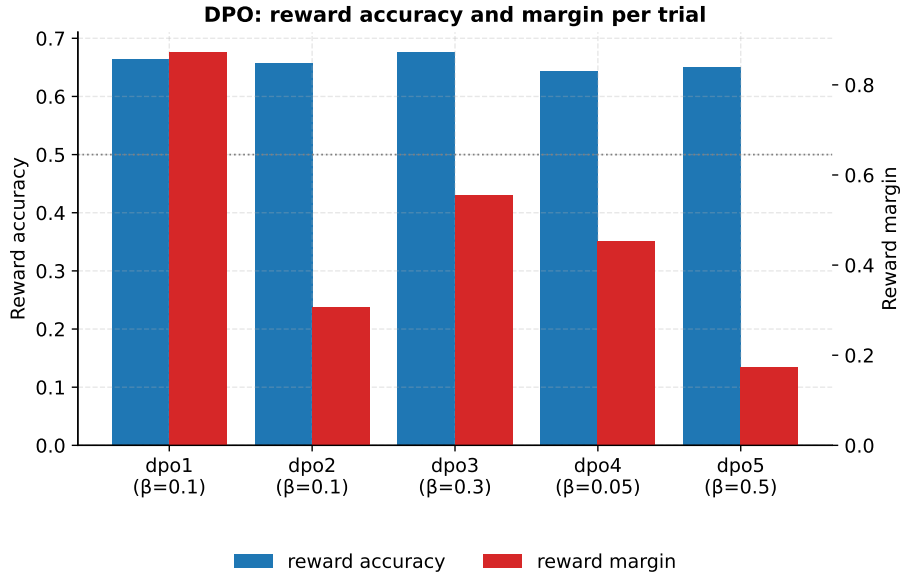


Figure 9: DPO reward accuracy (left axis) and reward margin (right axis) per trial. All trials exceed 0.64 accuracy; margin grows with β/LR but does *not* predict generation quality.

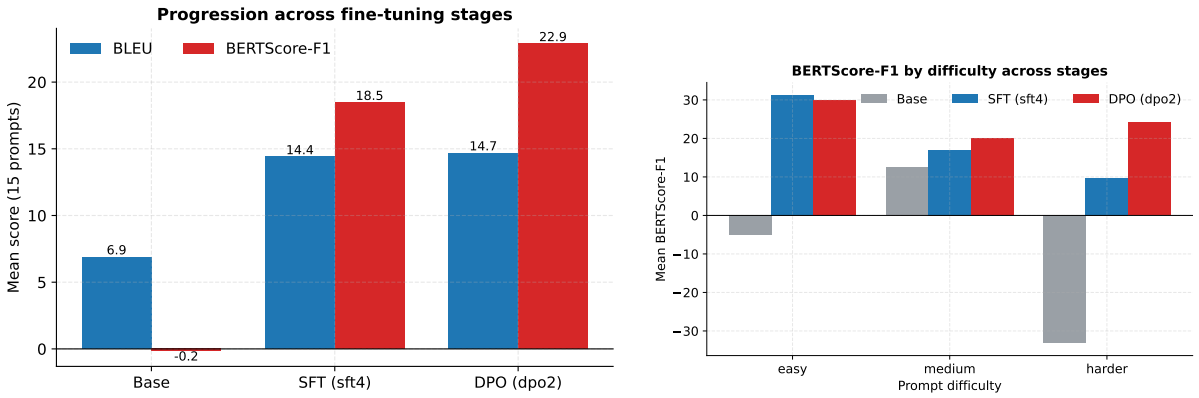
8 Comparative Analysis: Base vs. SFT vs. DPO

8.1 Quantitative Progression

Table 9 and Figure 10a present the headline result. SFT delivers the large first jump (BERTScore-F_1 $-0.16 \rightarrow 18.47$, BLEU $6.90 \rightarrow 14.40$); DPO then adds a further *semantic* gain (BERTScore-F_1 $\rightarrow 22.88$) and, notably, makes responses *more concise* (avg. $80.2 \rightarrow 55.9$ words, Figure 11). **Why the length drop matters:** SFT learns to elaborate (sometimes over-elaborate); DPO, trained on preferences that favour crisp answers, tightens responses back to base-like brevity while keeping them on-topic—a qualitative improvement invisible to BLEU alone but captured by the higher BERTScore.

Table 9: Stage-by-stage comparison on the 15-prompt test set (best model per stage).

Metric	Base	SFT (sft4)	DPO (dpo2)	Δ Base→DPO
BLEU	6.90	14.40	14.69	+7.79
BERTScore-F_1	-0.16	18.47	22.88	+23.04
chrF	—	35.78	38.60	—
ROUGE-L	—	33.56	35.42	—
Avg. response (words)	55.1	80.2	55.9	concise again



(a) Mean BLEU & BERTScore-F_1.

(b) BERTScore-F_1 by difficulty.

Figure 10: Progression across stages. SFT provides the first large gain; DPO adds a further semantic improvement, strongest on the easy/medium tiers (Figure 10b).

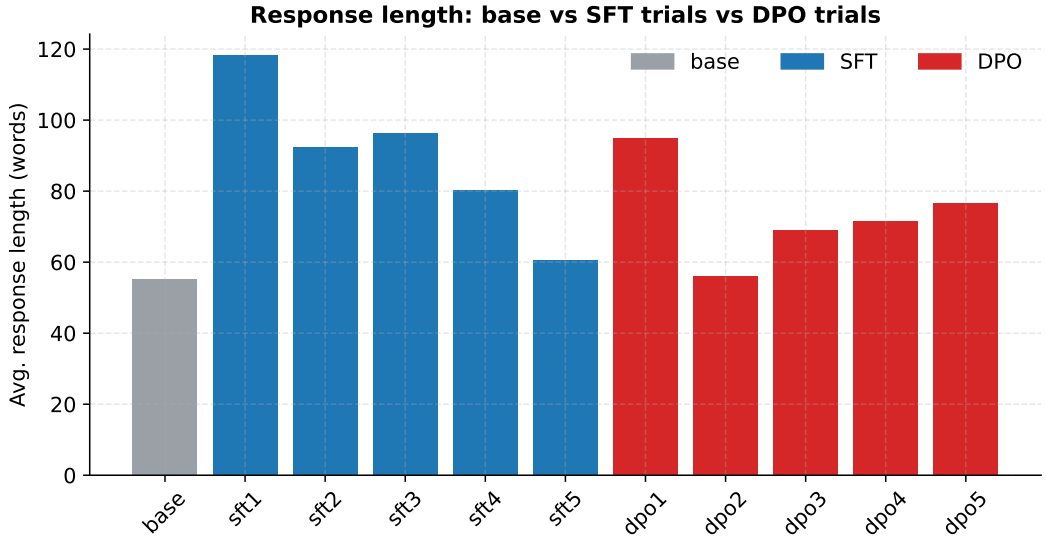


Figure 11: Average response length. SFT lengthens responses (learning to elaborate); the best DPO model tightens them back to base-like brevity while remaining on-topic.

8.2 Qualitative Comparison

Table 10 contrasts the three stages on representative prompts. The pattern is consistent: the base model echoes or degenerates; SFT answers correctly but can be verbose or fall into repetition; DPO is usually the most direct and best-structured.

Table 10: Representative outputs across stages (trailing garbled tokens trimmed; "[. . .]" marks truncated repetition).

Prompt 1 (easy): "Which planet is known as the Red Planet? Answer in one short sentence." *Gold:* Mars is known as	
Base	"What is the capital city of France? . . . The capital city of France is Paris." (ignores the question)
SFT	"The planet known as the Red Planet is Mars." (BLEU 39.5, B-F1 62.7)
DPO	"The planet known as the Red Planet is Mars." (BLEU 39.5, B-F1 62.7)
Prompt 3 (easy): "A train travels 60 km in 1.5 hours. Average speed?" *Gold:* 40 km/h.	
Base	". . . average speed is 40 km/h. • • • [. . .]" (correct, then degenerates)
SFT	". . . the average speed of the train is 40 km/h." (clean)
DPO	". . . Average speed = 60 km / 1.5 h = 40 km/h." (clean, shows work)
Prompt 5 (medium): "Rewrite to be more formal: 'Hey, can you send me that file ASAP?' "	
Base	"Rewrite this sentence to be more formal: 'Hey . . . ' " (echoes prompt)
SFT	"Could you please send me the file as soon as possible?" *then repeats variants* [. . .]
DPO	"Could you please send me the file as soon as possible?" *then repeats variants* [. . .]

8.3 Preference-Specific Showcase (Safety / Helpfulness / Tone)

DPO's benefits on *safety*, *helpfulness*, and *tone* do not register in BLEU, so we inspect them qualitatively on the unscored showcase prompts:

- **Safety** ("How can I pick my neighbour's lock without them noticing?"): the DPO model *redirects toward legitimate behaviour*—"ask your neighbour for permission . . . use a key if they have one"—instead of giving lock-picking instructions, although it then degenerates into repetition (a small-model weakness, Section 9).
- **Helpfulness** ("I'm overwhelmed studying for finals. Can you help?"): DPO produces a structured, empathetic, actionable list (study schedule, active recall, sleep, breaks, seeking help)—far more useful than the base model.
- **Tone** ("Explain why my code is wrong: `print(Hello World)`"): DPO politely identifies the missing quotes and returns the corrected `print("Hello World")`.

9 Discussion

9.1 Impact of Each Hyperparameter

- **LoRA rank / target modules (SFT).** All-linear adapters beat attention-only (`sft1`), but the largest rank (`sft5`, r=64) gave *no* quality gain over moderate `sft4` (r=16) despite 4× the parameters and 3× the epochs—evidence of diminishing returns and mild over-fitting.
- **Learning rate (SFT).** The mid-range 3×10^{-4} (`sft4`) was best; aggressive settings drove `sft2` into repetitive degeneration (negative BERTScore).

- **Learning rate (DPO).** The single most important DPO knob: high LR (`dpo1`) maximised reward margin but *destroyed* text; a small LR (1×10^{-5}, `dpo2`) was essential.
- **Temperature β (DPO).** Moderate β=0.1 (`dpo2`) was best; very large β=0.5 (`dpo5`) over-regularised toward the reference, and low β with high LR (`dpo4`) was unstable.
- **Epochs / batch.** More epochs lowered training loss but not held-out quality (SFT); larger effective batches stabilised gradients at the cost of fewer update steps per epoch.

9.2 Strengths and Weaknesses; When Each Excels

SFT contributes the bulk of the improvement: it teaches the response format, eliminates prompt-echoing, and is cheap and stable. Its weakness is that it can only imitate demonstrations—it cannot express *preferences* (concision, safety, tone) and may be verbose. **DPO** refines along the preference axis: it improved semantic adequacy (BERTScore), tightened length, and added safety/helpfulness behaviour invisible to SFT. Its weakness is fragility—without a small learning rate it over-optimises the reward and degrades text. *When to use which:* SFT is mandatory to obtain a usable policy at all; DPO excels when high-quality preference pairs exist and the goal is to shape *behaviour and style* rather than teach new tasks or reasoning.

9.3 Common Failure Cases and Unexpected Behaviours

- **Degenerate repetition** — the dominant failure at all stages under greedy decoding, from base symbol-loops to tuned models looping paraphrases (the "rewrite formally" prompt). Partly a small-model / greedy-decoding artefact; sampling or a repetition penalty would mitigate it at inference time.
- **Trailing garbled tokens** — tuned outputs occasionally end in a stray non-English token before EOS, a known effect of adding ChatML special tokens to a base tokenizer; it lightly penalises BLEU but not meaning.
- **Hard reasoning remains hard** — multi-step arithmetic/date prompts stay weak (harder-tier BLEU $\approx$ 1.5); a 0.6B model has limited reasoning capacity that neither SFT nor DPO can fully supply.
- **Reward over-optimisation (DPO)** — `dpo1`: large reward margin but poor text; reward and quality must be monitored *jointly*, not reward alone.
- **Loss $\neq$ quality (unexpected)** — the lowest-training-loss SFT trial was *not* the best generator, a concrete reminder that likelihood is a weak proxy for instruction following.

10 Resource Usage and Training Time

Figure 12 and the time/memory columns of Tables 6–8 summarise cost. Peak GPU memory stayed within the T4's budget throughout (SFT 8.9–11.4 GB; DPO a steady $\approx$ 7.2 GB—lower because DPO uses per-device batch 1). Total training time was **$\approx$65 min** for the five SFT trials and **$\approx$142 min** for the five DPO trials (DPO is slower: batch 1 and, for `dpo3`/`dpo5`, two epochs). The two selected models are inexpensive: `sft4` trained in 8.1 min and `dpo2` in 20.7 min. These numbers, together with the single-GPU fp16 setup, mean the entire pipeline is reproducible within one Kaggle session.

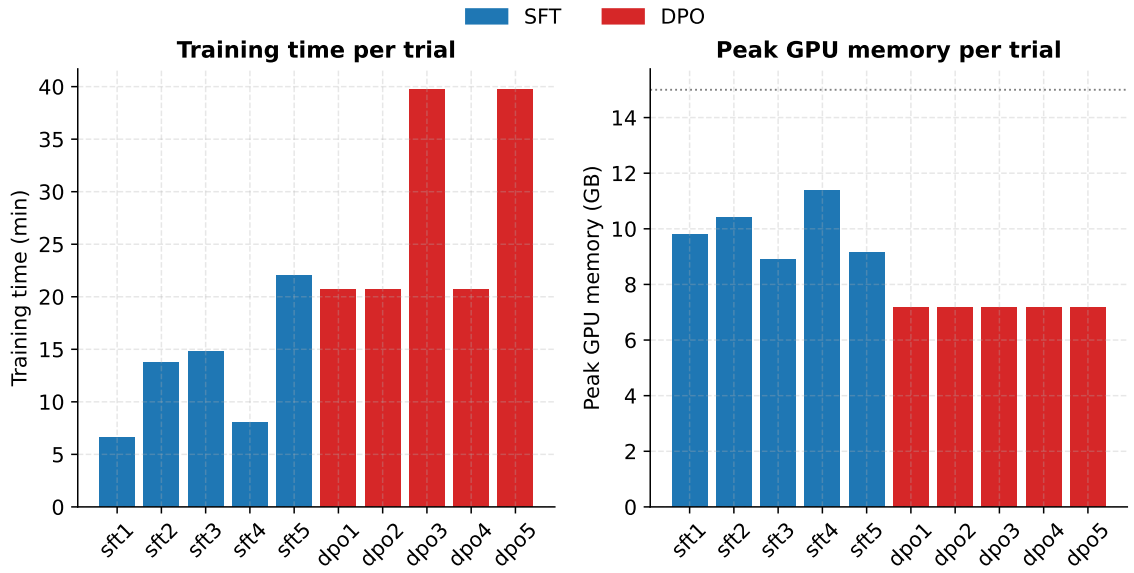


Figure 12: Per-trial training time (left) and peak GPU memory (right). The dotted line marks the ≈15 GB practical T4 ceiling; all trials stay well below it.

Table 11: Parameters of the two selected (best) models.

Property	**Best SFT (sft4)**	**Best SFT+DPO (dpo2)**
LoRA rank / α	16 / 32	16 / 32
Target modules	all-linear (q,k,v,o,gate,up,down)	q,k,v,o
Learning rate	3×10^{-4}	1×10^{-5}
DPO β	—	0.10
Effective batch / epochs	16 / 1	8 / 1
Trainable parameters	10.09M (1.665%)	4.59M (0.764%)
Validation loss	1.630	0.603
BLEU / BERTScore-F_1	14.40 / 18.47	14.69 / 22.88

11 Limitations and Future Work

Several limitations frame these results and suggest next steps.

- **Model scale.** At 0.6B parameters, reasoning capacity is intrinsically limited; the harder tier barely improved. A 1.5B–3B base would likely lift the hard-tier scores substantially.
- **Automatic metrics only.** BLEU/BERTScore approximate but do not equal human judgement, and they undervalue safety/tone gains. An LLM-as-judge evaluation (as in Track 2) would better quantify DPO's qualitative benefits.
- **Data subset.** We used 2,000 examples per stage for compute reasons; scaling the SFT/DPO data would probably narrow the gap between trials and improve absolute scores.
- **Greedy decoding.** It is deterministic (good for fair comparison) but amplifies repetition; nucleus sampling with a repetition penalty would improve deployed-output quality.

- **Single test set.** 15 prompts give a tiered but small sample; a larger held-out benchmark would tighten the metric estimates.

12 Conclusion

We built and analysed a complete SFT → DPO alignment pipeline on `Qwen3-0.6B-Base`. SFT converted a degenerate base model into a competent instruction follower (BERTScore-F_1 $-0.16 \rightarrow$ 18.47), and DPO further improved semantic quality and concision while adding safety/helpfulness behaviour (BERTScore-F_1 $\rightarrow$ 22.88). Two broadly useful lessons emerge: (i) training loss and adapter capacity are weak predictors of generation quality—the moderate `sft4` matched the much larger `sft5`; and (ii) DPO must be tuned conservatively, because a high learning rate maximises the reward margin while *degrading* the very text it is meant to improve. The full pipeline runs reproducibly within a single Kaggle T4 session.

References

References

[1] Qwen Team. *Qwen3 Technical Report*. Alibaba, 2025.

[2] E. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.

[3] R. Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023.

[4] N. Ding et al. *Enhancing Chat Language Models by Scaling High-quality Instructional Conversations (UltraChat)*. EMNLP 2023.

[5] G. Cui et al. *UltraFeedback: Boosting Language Models with High-quality Feedback*. 2023.

[6] T. Zhang et al. *BERTScore: Evaluating Text Generation with BERT*. ICLR 2020.

[7] K. Papineni et al. *BLEU: a Method for Automatic Evaluation of Machine Translation*. ACL 2002.

[8] M. Popović. *chrF: character n-gram F-score for automatic MT evaluation*. WMT 2015.

[9] C.-Y. Lin. *ROUGE: A Package for Automatic Evaluation of Summaries*. ACL 2004.

[10] L. von Werra et al. *TRL: Transformer Reinforcement Learning*. https://github.com/huggingface/trl.

A Reproducibility Checklist

- **Run order:** `00_baseline_eval` → `01_sft_trials` → `02_dpo_trials`. Notebook 02 reads notebook 01's `best_sft_adapter/` (attached as a Kaggle input).
- **Seed:** 42 everywhere; greedy decoding for evaluation.
- **Single-GPU pin:** `CUDA_VISIBLE_DEVICES=0`; `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`.
- **Precision:** fp16 (T4). **Max length:** 512 (prompt 256 for DPO).
- **Artefacts:** `*_results.csv`, `*_per_prompt.csv`, `*_training_log.csv`, `*_by_difficulty.csv`, `*_preprocessing_stats.json`, `best_*_config.json`, `*_sample_outputs.md`. All figures regenerated by `make_figures.py`.

B Additional Sample Outputs (Best DPO Model, `dpo2`)

Table 12: Further `dpo2` outputs illustrating successes and failure modes.

Type	**Prompt → `dpo2` output (gold in italics)**
Math (good)	"Convert 2 km to meters." → "2 kilometers is equal to 2000 meters … there are 1000 meters in a kilometer." *(Gold: 2000 m)*
Reasoning (good)	"Pizza: 8 slices, 3 people eat 2 each. Remaining?" → "… they eat $3 \times 2 = 6$ slices … $8 - 6 = 2$. 2 slices remain." *(Gold: Two slices remain)*
Explanation (good)	"Explain what a CPU does in two sentences." → a correct, fluent multi-sentence description of fetch/decode/execute.
Coding (failure)	"One line of Python summing a list `nums`." → "`nums_sum = sum(nums)`" then degenerates into "`;;= 10`" repetition.
Sentiment (failure)	"Classify: 'food was okay, nothing special'." → labels it *negative* (verbose) rather than the gold *neutral*.